Trainer Guide: Spring Boot Session 1 - Foundations

A slide-by-slide read-aloud script for springboot-session1-theory.html. Everything on screen, explained in plain spoken language.

Presenter: Purvam Prajapati - Faculty, AHD
Companion to: springboot-session1-theory.html (33 slides)

Each slide has an ON SCREEN reminder and a SAY section you can read out word-for-word. EMPHASISE and DELIVERY TIP notes are for you only - do not read those aloud.

Opening

Slide 1 - Title: Spring Boot

ON SCREEN

The blue cover slide: 'Spring Boot', with the line 'from zero to a running REST API'.

SAY (read aloud)

Welcome, everyone, to the first session in our Spring Boot series. Spring Boot is the framework that powers most modern Java backends, and over these sessions we'll go from a blank page to a real, running web service. Today is the foundation. We'll answer three questions in order: why does Spring Boot exist, how does it work under the hood, and how do we write our first code with it. By the end of this session you'll be able to explain what Spring Boot actually does for you, and you'll have created a running REST API from scratch in the lab.

DELIVERY TIP

Keep this light and welcoming. Many in the room may have heard 'Spring' is hard - reassure them that Boot exists precisely to make it easy.

Slide 2 - Agenda

ON SCREEN

Seven numbered agenda items, from why Spring Boot exists through to the hands-on labs.

SAY (read aloud)

Here's our path. We start with why Spring Boot exists - the pain of traditional Spring that it was built to remove. Then what Spring Boot actually is, where it sits, and how it works. We'll look at the request architecture and the four layers of an application. We'll unpack the one annotation that starts everything, at-SpringBootApplication. We'll cover dependency injection and the IoC container - core Spring ideas. Then the project itself: the Initializr tool, the folder structure, the pom file, and properties. And finally the hands-on lab plus a look ahead to Session 2. Spring is a large ecosystem, so think of today as the foundation that data access, security, and microservices all build on later.

Part 1 - Why Spring Boot Exists

Slide 3 - Life before Spring Boot

ON SCREEN

A bulleted list of the manual setup work required by classic Spring, and a warning box about the real cost.

SAY (read aloud)

To appreciate Spring Boot, you have to feel the pain it removed. The Spring Framework arrived in 2003 and was genuinely great - it introduced dependency injection and a clean way to structure code. But while Spring made well-organised code possible, simply starting and assembling a project was slow and error-prone. Before writing a single line of business logic you had to: write reams of XML to declare every object and how they connect; manually wire dependencies together; install and configure an external Tomcat server, build a WAR file, and deploy into it; hand-pick library versions and pray they were compatible; and then copy that same boilerplate into every new project. Teams routinely spent a day or more just getting an empty project to start up. Spring Boot was created in 2014 to eliminate exactly this friction.

Slide 4 - XML configuration hell

ON SCREEN

A block of classic Spring XML defining beans, plus bullet points on why it was painful.

SAY (read aloud)

Here's pain point number one, made concrete. In classic Spring, the container couldn't discover your objects automatically - you described every single one in XML. Look at this example: each bean is declared by hand, and you manually link one bean to another with property and ref tags. Three problems. Every new class meant another bean entry, maintained by hand. A typo in a class name or a reference wasn't caught by the compiler - it failed only at runtime, often deep into startup. And on real projects this XML grew into hundreds of lines, often bigger and harder to maintain than the Java it was wiring. Spring Boot replaces nearly all of this with annotations and auto-configuration - the XML becomes a couple of annotations the framework discovers for you.

Slide 5 - Wiring, servers & version juggling

ON SCREEN

Four cards: manual wiring, no embedded server, version conflicts, repeated boilerplate.

SAY (read aloud)

Pain points two, three, and four, as four cards. Manual wiring: you connected every dependency yourself, and a forgotten one only showed up when the app crashed at runtime. No embedded server: you built a WAR, installed Tomcat, configured it, and deployed into it - four slow manual steps before the app would even respond. Version conflicts: you aligned versions of Spring, Jackson, logging, and the servlet API by hand, and mismatches caused the infamous dependency hell. And repeated boilerplate: the same scaffolding copied into every project before any real work began. Notice the pattern behind all four - every one is configuration work that is the same for almost every project.

EMPHASISE

State the key insight: if ninety-five percent of projects configure these the same way, the framework should do it by default and let you override only the five percent that differs. That idea is called convention over configuration, and it is the heart of Spring Boot.

Part 2 - What Spring Boot Is

Slide 6 - What is Spring Boot?

ON SCREEN

A vertical stack: Your Application runs on Spring Boot, which is built on the Spring Framework. A 'mental model' key box.

SAY (read aloud)

So here's the solution. Spring Boot is a layer built on top of the Spring Framework that makes it easy to create stand-alone, production-grade applications you can simply run. The crucial point - and please stress this - is that Boot does not replace Spring. You still write Spring beans, controllers, and services exactly as before. Boot just removes the manual setup that used to surround them. Look at the stack on screen: your application sits on top, it runs on Spring Boot, and Boot is built on the Spring Framework underneath.

EMPHASISE

Give them the mental model: Spring is the engine; Spring Boot is the ignition and auto-tuning that gets the engine running instantly. Everything they learn about core Spring still applies - Boot just means time spent on features instead of setup.

Slide 7 - The four things Boot provides

ON SCREEN

Four cards: auto-configuration, embedded server, starter dependencies, production-ready.

SAY (read aloud)

Everything Spring Boot does fits into four capabilities, and understanding these four is understanding Spring Boot itself. One: auto-configuration - it inspects your classpath and configures sensible beans automatically, replacing the old XML. Two: an embedded server - Tomcat is built right into your application, so you run it as an ordinary Java program with no external server to install. Three: starter dependencies - one curated dependency brings in everything needed for a capability, with versions guaranteed to fit together. And four: production-ready features - health checks, metrics, and monitoring out of the box through a module called Actuator. Together, these turn 'a day of setup' into 'run the main method'. We'll look at each one over the next few slides.

Slide 8 - Convention over configuration

ON SCREEN

Bullet examples of Boot making intelligent guesses from the classpath, and a key box on day-to-day impact.

SAY (read aloud)

This is the philosophy that makes everything else possible. Rather than forcing you to describe every detail, Spring Boot assumes the most common sensible setup - the convention - and lets you override only what's genuinely different - the configuration. It makes intelligent guesses from what it can see. It sees the web starter on the classpath, so it assumes you're building a web app and starts Tomcat on port 8080. It sees an H2 database driver, so it assumes an in-memory database and configures the connection for you. It finds a class marked at-RestController, so it registers it to handle web requests automatically. What this means day to day: you write configuration only when your needs differ from the convention. If you're happy with the defaults - and usually you are - you write nothing at all.

Slide 9 - Auto-configuration, step by step

ON SCREEN

A three-box flow: scan classpath, check conditions, configure beans. Bullets on conditional activation.

SAY (read aloud)

Let's see how auto-configuration actually works, because this is the mechanism that replaced XML. At startup, Spring Boot runs a decision process across hundreds of pre-written configuration classes, each asking 'should I activate, given this particular project?' Follow the flow: first it scans the classpath to see which jars are present. Then it checks conditions - annotations like ConditionalOnClass and ConditionalOnMissingBean. Then it configures only the beans that are actually needed. So each auto-configuration is conditional - it activates only if the relevant library is present and you haven't already defined that bean yourself. This is exactly why simply adding a starter is enough to get working features. And importantly, if you define your own bean of the same type, Boot backs off and uses yours.

DELIVERY TIP

Mention the debugging trick: running with dash-dash-debug prints an auto-configuration report showing what matched and why. Useful when something isn't configured as expected.

Slide 10 - The embedded server

ON SCREEN

Bullets on the fat JAR, plus a before/after comparison of traditional WAR deployment versus Spring Boot.

SAY (read aloud)

Traditionally, a Java web app was a WAR file deployed into a separately installed server. Spring Boot flips this: the web server is packaged inside your application, so there's nothing external to install or deploy into. Running the main method starts your application and its own web server in one step. Everything packages into a single executable 'fat' JAR - your code, all its dependencies, and the server, in one file. You deploy by copying that one file to any machine with Java and running 'java dash jar app dot jar'. Tomcat is the default, and switching to Jetty or Undertow is a one-dependency change. The before-and-after on screen says it all: four manual steps become 'build one JAR, the server's inside it, run it, done'.

Slide 11 - Starter dependencies

ON SCREEN

The starter-web XML snippet, then a flow showing one line pulling in MVC, Jackson, Validation, and Tomcat.

SAY (read aloud)

A starter is a curated, ready-made bundle of dependencies for one capability. Instead of researching and version-matching a dozen separate libraries, you declare a single starter and trust Spring Boot to bring in everything that belongs together. Look at the snippet - that one dependency, spring-boot-starter-web, pulls in Spring MVC, the Jackson JSON library, validation, and an embedded Tomcat, all at once, as the flow shows. And here's why this ends dependency hell: the versions inside a starter are chosen and tested by the Spring team to work together. You don't specify versions at all - they're inherited from the Spring Boot parent. Upgrade Boot once, and every managed library moves together, safely.

Slide 12 - Common Spring Boot starters

ON SCREEN

A table of the starters they'll meet most: web, data-jpa, security, test, thymeleaf, validation.

SAY (read aloud)

There are dozens of official starters, all following the naming pattern spring-boot-starter-something. These are the ones you'll meet most. starter-web for web apps and REST APIs - that's today's one. starter-data-jpa for database access, which we use in Session 2. starter-security for authentication and authorization. starter-test, which brings JUnit, Mockito, and Spring Test, and is added automatically. starter-thymeleaf for server-rendered HTML pages. And starter-validation for input checking. The pattern to take away: you add starters as your app grows. Need a database next session? Add the JPA starter. Today we use only the web starter.

Slide 13 - Production-ready features

ON SCREEN

Bullets on Actuator endpoints - health, metrics, info - and a note that it's covered later.

SAY (read aloud)

Building an app is only half the job - in production you must observe and manage it. Is it healthy? How much memory is it using? How many requests is it handling? Spring Boot's Actuator module adds these management endpoints with almost no effort. Slash-actuator-slash-health reports whether the app and its dependencies are healthy - load balancers poll this to decide whether to send traffic. Slash-actuator-slash-metrics gives memory, CPU, garbage collection, and request timings. Slash-actuator-slash-info gives build and version details. These feed monitoring tools like Prometheus and Grafana that ops teams rely on. We won't configure Actuator today - the point for now is conceptual: production concerns are built into Spring Boot, not bolted on later. That's what 'production-grade' really means.

Part 3 - Architecture & How It Starts

Slide 14 - How a request flows

ON SCREEN

A horizontal architecture diagram: Browser, HTTP Request, DispatcherServlet, Controller, Service, Repository, Database.

SAY (read aloud)

This is the most important picture today, so let's trace a request across it. A browser sends an HTTP request, say GET slash hello. It arrives at the DispatcherServlet - Spring's front controller. The DispatcherServlet routes it to the correct controller method based on the URL. The controller calls a service for the business logic. The service calls a repository for data. And the repository talks to the database. The response then travels back along the very same path. The key idea: one central DispatcherServlet receives every incoming request and dispatches it to the right place. Each box has exactly one job.

EMPHASISE

Name the principle: this is separation of concerns. Each component does one thing, which is what keeps applications testable and easy to change. Keep this diagram in mind - we revisit it all series.

Slide 15 - Responsibilities by layer

ON SCREEN

A stacked diagram of four layers: Controller, Service, Repository, Database, each with its responsibility.

SAY (read aloud)

Let's name the layers and their single responsibilities. The Controller receives the HTTP request, reads the input, calls a service, and returns a response - it knows about the web but contains no business rules. The Service is the brains: business logic, calculations, validation, decisions - it knows nothing about HTTP or how data is stored. The Repository talks to the database - save, find, update, delete - pure data access. And the Database stores the data so it survives restarts. Why split it up? Because each layer talks only to the one directly below it, you can change one without breaking the others - swap the database, redesign the API, or unit-test the service in isolation. Today's lab builds the Controller and Service layers; the Repository and Database arrive in Session 2.

Slide 16 - @SpringBootApplication

ON SCREEN

The main class with @SpringBootApplication and the SpringApplication.run line, plus four bullets of what run() does.

SAY (read aloud)

Every Spring Boot application has exactly one main class carrying this annotation, and running its ordinary Java main method is what boots the whole application to life. Look at the code - it's tiny. The single line, SpringApplication dot run, does a remarkable amount of work behind the scenes, in order: it creates the IoC container, the application context; it runs auto-configuration based on the classpath; it scans your packages for components and registers them as beans; and it starts the embedded Tomcat server. By the time main returns, your API is live and listening on port 8080. One annotation, one line - and an entire web application is running.

Slide 17 - @SpringBootApplication, unpacked

ON SCREEN

Three rows mapping the bundled annotations: Configuration, EnableAutoConfiguration, ComponentScan. A warning about package placement.

SAY (read aloud)

It looks like magic, but this annotation is simply a convenience that bundles three others - and knowing them removes the mystery. At-Configuration marks the class as a source of bean definitions. At-EnableAutoConfiguration switches on the auto-configuration we discussed, based on the classpath. And at-ComponentScan scans this class's package and all sub-packages for components - controllers, services, repositories - and registers them as beans.

EMPHASISE

Stress the package-placement rule - it's one of the most common beginner bugs. Because ComponentScan starts at the main class's package and scans downward, your controllers and services must live in that package or a sub-package. Put a controller somewhere else and Spring never finds it - the endpoint silently returns 404 with no error.

Part 4 - Dependency Injection & IoC

Slide 18 - Dependency Injection: the why

ON SCREEN

Bullets on the benefits of DI - looser coupling, easier testing, less plumbing, single responsibility - and a one-line definition.

SAY (read aloud)

Almost every object needs other objects to do its job - a controller needs a service, a service needs a repository. The question is: who creates those collaborators? Dependency Injection says an object should not create its own collaborators. Instead they're created elsewhere and injected - handed in - from outside. Why is that better? Looser coupling - a class depends on what it needs, not on how that thing is built. Easier testing - in a test you can inject a fake or mock instead of the real thing. Less plumbing - Spring's container creates the objects and supplies them automatically. And single responsibility - a class focuses on its own job, not on managing its dependencies' lifecycles.

EMPHASISE

Give them the memorable one-liner on screen: 'Don't call us, we'll call you.' Your class declares what it needs; the framework provides it. That reversal of control is exactly what Inversion of Control means.

Slide 19 - Tight vs loose coupling

ON SCREEN

Two code panels: tight coupling using 'new', versus loose coupling using @Autowired. A real-world payoff note.

SAY (read aloud)

Let's make coupling concrete with the two panels. On the left, tight coupling: the controller writes 'new GreetingService'. It is now permanently glued to that one exact class - you can never substitute a different implementation, or a mock for testing, without editing the controller's source. On the right, loose coupling: the service is injected with at-Autowired. The controller just asks for 'a GreetingService' and Spring decides what to provide. You can swap implementations, wrap them, or mock them freely - the controller never changes. The real-world payoff: suppose you later need to log every greeting, or fetch greetings from a database. With loose coupling you just provide a different bean and change nothing in the controller. Loose coupling is what makes large codebases maintainable.

Slide 20 - Components, stereotypes & @Autowired

ON SCREEN

Bullets on @Component and @Autowired, then a table of stereotype annotations: RestController, Service, Repository, Component.

SAY (read aloud)

For Spring to inject a bean, it must first know about that bean - you tell it by annotating your classes. There are two halves: registering a bean, and requesting one. At-Component says 'Spring, please create and manage an instance of this class' - during scanning it becomes a bean in the container. At-Autowired says 'Spring, inject a matching managed bean right here.' Now the table shows the stereotype annotations, which are all specialised forms of at-Component. They register beans the same way, but each signals the class's role, which makes code self-documenting. At-RestController for the web layer, at-Service for business logic, at-Repository for data access - which also translates database exceptions - and plain at-Component for anything else Spring should manage.

Slide 21 - The IoC container

ON SCREEN

A dashed 'Spring IoC Container' box holding three beans, with a note that Spring injects them where @Autowired asks.

SAY (read aloud)

Inversion of Control is the principle; the IoC container - also called the application context - is the part of Spring that implements it. Instead of your code controlling when and how objects are created, you hand that responsibility to the container. At startup it scans for your annotated components, creates one instance of each - a singleton by default - works out what each one depends on, and injects those dependencies automatically. Look at the diagram: the container holds the controller, the service, and the repository beans. In our app it creates the GreetingService once, sees that HelloController needs a GreetingService, and hands the same instance over. You never wrote 'new' - that's the inversion: the framework assembles your code, rather than your code assembling the framework.

Slide 22 - Field, constructor & setter injection

ON SCREEN

Two code panels - constructor injection (recommended) and field injection (simplest) - plus bullets on the three styles.

SAY (read aloud)

Spring can inject a dependency in three places, and the trade-offs are worth knowing. Constructor injection, on the left, is the modern best practice: the dependency is passed into the constructor and stored in a final field. It can never be null and never change after construction, the class is easy to instantiate in tests, and required dependencies are obvious. With a single constructor, at-Autowired is even optional. Field injection, on the right, is the shortest - Spring sets the field directly. We use it in today's lab purely for clarity, but it's harder to test and hides required dependencies. And setter injection - Spring calls a setter - is best for optional dependencies that can change after the object is built.

DELIVERY TIP

In the lab you'll use field injection for simplicity, but tell them clearly that in real projects they should prefer constructor injection.

Part 5 - REST & The Project

Slide 23 - What is REST?

ON SCREEN

Bullets defining resources, verbs, statelessness, and JSON; a small flow of a GET request and response.

SAY (read aloud)

REST - Representational State Transfer - is the dominant architectural style for web APIs. The core idea is to model your application as a set of resources - nouns - each addressable by a URL, and to act on them using the standard HTTP methods - verbs. A resource is a thing: a user, an order, a greeting. It lives at a URL like slash-users-slash-42. You act on it with a verb - GET to read, POST to create, PUT to update, DELETE to remove. Each request is stateless - it carries everything the server needs, so the server keeps no memory of previous requests, which is what makes REST easy to scale. Responses are usually JSON, though today we'll return plain text to keep our first example simple.

Slide 24 - HTTP methods & mappings

ON SCREEN

A table mapping GET, POST, PUT, DELETE to their Spring annotations, and a note on today vs Session 2.

SAY (read aloud)

Every interaction with a REST API uses an HTTP method that expresses intent, and Spring gives each verb a dedicated annotation that maps a URL to one of your Java methods. GET reads data and is safe - it makes no changes - and maps to at-GetMapping. POST creates new data, at-PostMapping. PUT updates or replaces, at-PutMapping. DELETE removes, at-DeleteMapping. Choosing the right verb is part of designing a clean API - for instance, a GET must never modify data, so it's safe to retry or cache. Today's lab uses only at-GetMapping, because reading data is the simplest case - you test it just by visiting a URL in your browser. The other three, which change data, come with the database in Session 2 when we build full CRUD.

Slide 25 - Spring Initializr

ON SCREEN

A table explaining each field on start.spring.io and what we choose: Maven, Java, Group, Artifact, Jar, Java 17, web starter.

SAY (read aloud)

You never start a Spring Boot project from an empty folder. Spring Initializr is an official web tool - also built into IntelliJ - that generates a complete, ready-to-run project skeleton with the right structure, build file, and dependencies. Let's read the fields. Project is the build tool - we choose Maven. Language - Java. Spring Boot - the latest stable version. Group is the reverse-domain package root - com dot tcs dot training. Artifact is the project name - springboot-demo. Packaging - Jar, executable and self-contained, not War. Java - version 17, a long-term-support release. And Dependencies - just spring-boot-starter-web for now. Clicking Generate downloads a ZIP - that's your starting project, which you import into the IDE.

Slide 26 - Project structure

ON SCREEN

A folder tree of the generated project, with bullets and a key box about what's missing.

SAY (read aloud)

When you unzip the project you'll find this layout - standard Maven convention, so every Spring Boot project looks familiar once you know it. src-main-java holds your application code, including the main class. src-main-resources holds configuration and static files, including application dot properties. src-test-java holds your tests. The pom dot xml declares dependencies and the build. And mvnw is the Maven wrapper, which lets you run Maven without installing it. The main class sits at the top of your package so component scanning reaches everything beneath it.

EMPHASISE

Point out what's missing, because that absence IS Spring Boot's value made visible: there's no web dot xml, no tomcat folder, and no XML configuration anywhere. The server is embedded and configuration is automatic.

Slide 27 - pom.xml anatomy

ON SCREEN

A pom.xml snippet showing parent, dependencies, and the Spring Boot Maven plugin.

SAY (read aloud)

The pom dot xml - Project Object Model - is Maven's build file. It declares what your project depends on and how it's built. A Spring Boot pom has three parts worth knowing. First, the parent: your project inherits from spring-boot-starter-parent, which provides all those tested, compatible library versions, so you rarely specify a version yourself. Second, dependencies: this is where your starters go - here, just the web starter. Third, the build plugin: the spring-boot-maven-plugin is what packages everything into that single executable fat JAR we talked about. So the pom is short, because the parent and the starters are doing the heavy lifting for you.

Slide 28 - application.properties

ON SCREEN

Examples of common properties - server.port, logging levels - and the idea of overriding conventions.

SAY (read aloud)

This file, in src-main-resources, is where you override Spring Boot's conventions when you need to. Remember: if you're happy with the defaults you can leave it almost empty. Common entries: server dot port equals 9090 changes the port from the default 8080. You can set logging levels, the application name, database connection details, and so on. Each line is a simple key-equals-value. The mental model to leave them with: Spring Boot already made sensible choices for you; application dot properties is simply where you say 'except for this one thing, do it my way.' There's also a YAML form, application dot yml, which expresses the same settings in a nested style - they'll see both in the wild.

Slide 29 - Common mistakes

ON SCREEN

A list of frequent beginner errors - controller outside the scanned package, missing annotations, port conflicts.

SAY (read aloud)

Let's pre-empt the bugs you'll most likely hit, so you recognise them instantly. First and most common: a controller placed outside the main class's package, so component scanning misses it and your endpoint returns 404 with no obvious error. Keep controllers in a sub-package of the main class. Second: forgetting the at-RestController or at-Service annotation, so Spring never registers the bean. Third: a port conflict - 'port 8080 already in use' - because another app, or a previous run, is still holding it; stop it or change the port. And fourth: expecting an injected field to work when the class itself wasn't created by Spring - injection only happens for Spring-managed beans. If something 'just doesn't work', check these four first.

DELIVERY TIP

Share a war story if you have one - a real 'I lost an hour to the package-scan 404' anecdote makes this stick far better than the bullet list.

Lab, Recap & Close

Slide 30 - Lab preview

ON SCREEN

A preview of what they'll build in the hands-on lab: a HelloController and a GreetingService.

SAY (read aloud)

Now to the hands-on part. In the lab you'll create your first Spring Boot project from the Initializr, then build two classes that mirror the layers we discussed. A GreetingService, marked at-Service, holds the business logic - it returns a greeting message. And a HelloController, marked at-RestController, exposes a GET endpoint and calls the service to get its message. You'll inject the service into the controller, run the main method, and hit the endpoint in your browser to see your greeting come back. It's small on purpose - the goal is to see the whole machine work end to end: a request comes in, the controller delegates to the service, and a response goes out.

Slide 31 - Recap

ON SCREEN

A recap checklist of the session's key takeaways.

SAY (read aloud)

Let's recap what you now know. Spring Boot is a layer on top of Spring that removes setup friction through four things: auto-configuration, an embedded server, starter dependencies, and production-ready features. Its guiding philosophy is convention over configuration. A request flows through the DispatcherServlet to a controller, then a service, then a repository, then the database - separation of concerns. At-SpringBootApplication bundles three annotations and its run method boots everything. Dependency injection and the IoC container mean the framework creates and wires your objects, giving you loose coupling and testability. And REST models resources as URLs acted on by HTTP verbs. That's the foundation - everything else in the series builds on it.

Slide 32 - Coming up in Session 2

ON SCREEN

A preview of Session 2: building a full REST web service with CRUD.

SAY (read aloud)

A quick look ahead. Today our app simply ran and returned a greeting. In Session 2 we make it genuinely useful - we turn it into a full REST web service that exchanges JSON data with any client. We'll build complete CRUD - Create, Read, Update, and Delete - using all four HTTP verbs, learn how Java objects become JSON automatically, work with path variables and request bodies, and test it all with Postman and Swagger. The theory will be shorter next time, because the real weight of Session 2 is the hands-on lab where you build the whole API yourself.

Slide 33 - Closing

ON SCREEN

The closing blue slide.

SAY (read aloud)

And that's our foundation. You can now explain what Spring Boot does for you and why it exists, and in the lab you'll have a running REST API to prove it. Let's open the lab sheet and build your first Spring Boot application together. Thank you - and let's take any questions first.

DELIVERY TIP

Common questions here: 'do I need to learn core Spring first?' - no, Boot teaches it as you go; and 'why not just use the defaults forever?' - you mostly do, properties is only for the exceptions.